

CPU 数据通路与控制

从 ISA 状态差到可计时、可验证的精确提交

408 · 计算机组成原理 Topic CO-03

母问题：一条指令承诺的状态变化，怎样由有限硬件真正发生？

ISA 只规定软件应当观察到什么变化；数据通路和控制器必须决定这些变化由哪些值、部件、路径和时钟边界实现。本册的生成链是

ISA State Delta $\rightarrow$ Required Values $\rightarrow$ Producer-Consumer Graph $\rightarrow$ Datapath $\rightarrow$ Resource Schedule $\rightarrow$ Cont

箭头表示设计依赖：后一步必须能够实现前一步，而不能反过来用某张教材通路图篡改指令语义。

本册学习范围

- 本册从一条指令前后的架构状态差出发，推导寄存器、总线、算术逻辑单元、存储接口和控制信号怎样协同工作。
- 本册重点解释微操作、单周期/多周期、单总线、硬布线控制、微程序控制、关键路径和精确提交。
- 流水线、Cache、TLB 和操作系统服务只作为接口条件出现；它们不会替代本册对单条指令内部时序的解释。

1 术语先行：把电路动作说成可检查的句子

本册的八个关键词

- **数据通路**：值从寄存器、ALU、存储器等部件流向另一个部件的实际路线。
- **控制器**：根据当前指令、阶段状态和条件信号，产生本拍允许的控制信号。
- **微操作**：一个时钟拍内完成的最小寄存器传送、运算或读写动作，例如 $R_1 \leftarrow R_2$ 。
- **控制字**：把多个控制信号打包后的编码，规定本拍谁驱动、谁接收、ALU 做什么以及下一状态。
- **MUX (多路选择器)**：从多个输入中选一个送往同一输出的电路。
- **FSM (有限状态机)**：用有限个状态和转移条件描述控制器随时钟推进的逻辑。
- **关键路径**：一拍内从输入稳定到结果可锁存的最长组合路径，它决定时钟周期下限。
- **提交**：在时钟边界把结果写入软件可见的寄存器、PC、内存或状态寄存器；临时寄存器里的值还不算提交。

目录

1 术语先行：把电路动作说成可检查的句子	1
2 为什么“取指、译码、执行”还不是模型	3
3 两类状态：软件契约与内部实现	3
3.1 架构状态	3
3.2 微架构状态	3
3.3 可访问性不能脱离 ISA 断言	4
4 从状态差生成值依赖图	4
4.1 四条母指令	4
4.2 LOAD 的生产者-消费者图	4
5 数据通路的最小构件	5
6 微操作：把依赖图调度到时钟边界	5
7 单周期与多周期：同一依赖图的两调度	6
7.1 单周期实现	6
7.2 多周期实现	6
7.3 公平比较	6
8 控制器：把条件映射为控制字与下一阶段	7
8.1 硬布线控制	7
8.2 微程序控制	7
9 提交：写使能才是架构状态变化的闸门	8
10 概念边界：相似名词为什么会误导通路题	8
11 贯穿母例：在单总线教学机上实现 LOAD	9
11.1 先写语义，不先写节拍	9
11.2 再按资源约束调度	9
12 怎样为新指令扩展数据通路	10
13 性能：从路径和资源生成数字	11
13.1 关键路径	11
13.2 资源复制与复用	11

13.3 题目成本模型	11
14 做题控制协议	11
14.1 题目信号与第一动作	12
15 高频失效模式	12
16 本册与指令语义的接口	12
17 考纲映射与接口	13
18 一页压缩：忘掉整张图后怎样重建	13
19 补充细节：从 CPU 功能到控制器状态机	14
19.1 CPU 的功能由程序状态变化反推	14
19.2 四个子周期与控制器的阶段证据	14
19.3 单总线数据通路：唯一驱动与中间寄存器	15
19.4 单周期与多周期：同一依赖图的两种调度	15
19.5 硬布线控制：从条件表生成控制逻辑	16
19.6 微程序控制：把控制字放入控制存储器	16
19.7 异常、中断与精确提交：控制器必须保护状态边界	16
19.8 教学机与模拟器应怎样使用	16
20 微架构陌生题反向重建协议	17

2 为什么“取指、译码、执行”还不是模型

阶段名只能描述大致顺序，不能回答以下问题：

- 哪一个值由谁产生，下一步由谁消费？
- 两个动作能否同拍发生，还是争用同一条总线或 ALU？
- 一个中间结果已经算出，是否已经改变软件可见状态？
- 控制器为何打开这个写使能，又为何必须关闭另一个写使能？

真正可生成通路的起点不是阶段名，而是指令的**状态差**。设软件可见架构状态为

$$A = (PC, R, M, F, \dots),$$

其中 R 表示架构寄存器集合， M 表示内存可见状态， F 表示架构标志或条件状态。一条指令定义状态转移

$$A_{t+1} = \mathcal{I}(A_t).$$

例如抽象指令

ADD rd, rs1, rs2

承诺的主要状态差是

$$R'[rd] = R[rs1] + R[rs2], \quad PC' = PC + L,$$

其余架构状态保持不变。这里的 L 是该 ISA 规定的顺序取指步长，不应无条件写死为 4。

第一条禁推关系

教材图中出现 PC 加法器、MAR、MDR 或某个固定节拍，不表示 ISA 要求所有实现都拥有这些具体部件。ISA Own “必须发生什么”；本册 Own “一种实现怎样保证它发生”。

3 两类状态：软件契约与内部实现

3.1 架构状态

架构状态（architectural state）是 ISA 允许软件观察或依赖的状态，例如通用寄存器、PC、架构标志、可见内存效果和规定的控制寄存器。处理器完成一条指令后，必须给出符合 ISA 的下一状态。

3.2 微架构状态

微架构状态（microarchitectural state）服务于实现，例如 IR、MAR、MDR、临时寄存器、控制状态寄存器、流水寄存器和 Cache 内部元数据。它们可以随实现不同而变化，但不能让软件看到违反 ISA 的结果。

internal states may differ $\wedge$ architectural trace must remain ISA-equivalent

同一条 ADD 的两种实现

实现甲在一个长周期内完成寄存器读取、加法和写回；实现乙先把两个源值锁存到临时寄存器，下一拍相加，再下一拍写回。两者内部状态轨迹不同，但只要提交后的 $R[rd]$ 、PC 和异常语义一致，就实现了同一条 ISA 指令。

3.3 可访问性不能脱离 ISA 断言

寄存器可以按软件能否直接读写分为“用户可见、部分可见、系统可见、完全不可见”，但具体归类依赖 ISA：

- PC、标志寄存器能否显式读写，取决于指令集；
- 页表基址、Cache 管理和 MMU 控制可能存在特权接口；
- PCB 指针通常是 OS 软件状态，不是必然存在的 CPU 专用寄存器；
- “对普通用户透明”不等于“对 OS 或固件完全不可见”。

稳定判据不是背清单，而是问：当前 ISA 是否提供了读取、写入、配置或间接影响该状态的合法操作？

4 从状态差生成值依赖图

对任意指令，先列四类量：

1. **Read Set**：完成语义必须读取哪些架构值？
2. **Derived Values**：中间必须产生哪些值？
3. **Write Set**：哪些架构位置允许改变？
4. **Next PC / Exception**：正常路径和异常路径怎样结束？

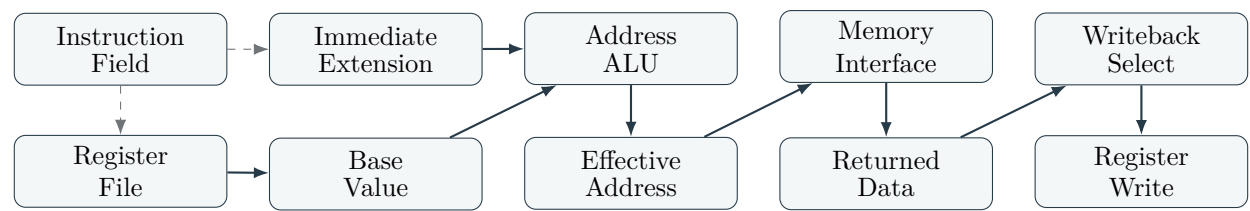
4.1 四条母指令

指令族	Read Set	Derived Values	Write Set	关键禁写
ADD	两个源寄存器、PC	算术结果、顺序 PC	目标寄存器、PC	内存不得被写
LOAD	base、立即数、PC	有效地址、返回数据、扩展值	目标寄存器、PC	内存内容不得被改
STORE	base、data、立即数、PC	有效地址、写数据、byte mask	指定内存位置、PC	通用目标寄存器不得误写
BRANCH	源值、PC、偏移	条件、目标 PC、顺序 PC	PC，可能还有架构约定状态	错路指令不得提交

4.2 LOAD 的生产者-消费者图

抽象 LOAD 可写成：

$$EA = R[base] + \text{ext}(imm), \quad D = M[EA], \quad R'[rd] = \text{ext}_{width}(D).$$



这张图的箭头表示数据依赖，不先承诺在哪一拍执行。先得到依赖图，再选择资源和调度，能够避免“照着已有图猜控制信号”。

5 数据通路的最小构件

构件	它提供什么能力	它不自动保证什么	常见约束
状态元件	跨时钟边界保存值	保存的值一定属于架构状态	setup/hold、写使能、复位
组合逻辑	在当前输入上计算函数	结果已经提交	传播延迟、扇出、关键路径
MUX	从候选来源中选一路	被选来源语义正确	选择信号、输入数、额外延迟
寄存器堆	按编号读写架构寄存器	所有实现都同拍两读一写	端口数、读写同址语义
存储接口	发请求并收发数据	请求一定本拍返回	latency、ready/valid、异常
总线	多部件共享传输媒介	多个源可同时驱动	仲裁、唯一驱动、带宽
控制器	产生使能、选择和下一阶段	数据值本身正确	输入条件完整、无非法组合

数据通路和控制通路的分工

Datapath: what values can move and transform Control: which legal movement happens now

数据通路提供可能性集合；控制器按指令、阶段和条件选出当前合法转移。

6 微操作：把依赖图调度到时钟边界

微操作是在一个受控阶段内发生的寄存器传送或组合变换。例如单总线教学机中：

$$\begin{aligned} R_1 &\rightarrow Y, \\ Y + R_2 &\rightarrow Z, \\ Z &\rightarrow R_3. \end{aligned}$$

能否同拍执行，不由语句长短决定，而要同时通过四道门：

1. **Dependency Gate:** 所有输入在本拍开始或组合路径要求的时刻已经可用；
2. **Resource Gate:** 总线、端口、ALU、存储口没有互斥占用；
3. **Timing Gate:** 最长组合路径满足时钟约束；
4. **Commit Gate:** 写使能不会产生 ISA 不允许的副作用。

对同步电路，一条基本时序约束可写为

$$T_{clk} \geq T_{cq} + T_{comb,max} + T_{setup} + T_{skew}.$$

题目若采用简化模型，可以省略某些项，但必须先声明成本口径。

单总线规则

同一时刻通常只允许一个源驱动总线；多个目标可以锁存同一个总线值。两条箭头画得下，不代表两个不同源能够同时输出。

7 单周期与多周期：同一依赖图的两调度

7.1 单周期实现

单周期处理器把一条指令的完整依赖链放进一个长组合周期。其 CPI 通常按教学模型取 1，但时钟周期必须覆盖最慢合法路径。

$$T_{clk}^{single} \geq \max_{i \in \text{instructions}} T_{path}(i).$$

它的核心代价不是“控制简单”四个字，而是：

- 短指令也被最长指令拖住；
- 同一拍重复使用某资源时，必须复制资源、增加端口或改变抽象；
- 组合路径越长，频率越受限。

7.2 多周期实现

多周期处理器在阶段边界加入临时状态，把依赖图切成多个较短周期，并允许不同阶段复用同一资源：

$$T_{instr}(i) = C_i T_{clk}^{multi}.$$

其中 C_i 是指令 i 的周期数。周期数变大并不自动变慢，因为 T_{clk}^{multi} 通常更短。

7.3 公平比较

$$T_{CPU} = IC \times CPI \times T_{clk}$$

比较实现时必须使用同一程序、同一指令统计口径，并同时考虑指令混合、周期时间和 CPI。单独比较“CPI=1”或“频率更高”都不足以判快慢。

维度	单周期	多周期
调度单位	一条指令一个长周期	一条指令多个短阶段
中间状态	尽量不跨拍保存	用 IR/MDR/临时寄存器保存
资源策略	复制或增加端口以消除同拍冲突	分时复用资源
控制输入	主要是指令与条件	指令、当前阶段与条件
主要风险	最长路径拖慢全部指令	状态机、暂存与控制更复杂
性能判断	$1 \times T_{clk}^{single}$	$C_i \times T_{clk}^{multi}$

不能从名称推出的结论

“单周期必须采用哈佛结构”“寄存器堆读端口永远无需使能”“LOAD 必然是最长路径”都只能在给定教学数据通路和存储时序下成立。稳定正文保留资源冲突与关键路径判据，不把某个实现选择升级为体系结构定律。

8 控制器：把条件映射为控制字与下一阶段

控制器解决的统一问题是

$$(instruction, phase, condition) \mapsto (control\ word, next\ phase)$$

控制字至少会包含三类决策：

- **Enable**：哪些状态元件允许写入；
- **Select**：MUX、ALU 和数据来源选哪一路；
- **Command**：存储读写、异常请求、阶段推进等动作。

8.1 硬布线控制

硬布线控制用组合/时序逻辑直接实现上述映射。它没有“不要经过任何状态”这一要求：多周期硬布线控制通常仍然是一个 FSM。其优势是避免逐条读取控制存储的开销，代价是复杂逻辑的设计、验证和修改成本。

8.2 微程序控制

微程序控制把控制字和下一微地址组织为微指令，存放在控制存储器中。机器指令触发某段微程序入口，微顺序器按条件选择后继微指令。

维度	硬布线	微程序	稳定判据
映射载体	逻辑网络/FSM	控制存储 + 微顺序器	都必须生成同一合法控制效果
速度来源	避免控存读取链	结构规整、便于复杂序列	具体快慢取决于实现与技术
修改成本	逻辑重设计成本高	修改微码相对集中	不等于 ISA 可随意修改
常见误区	“只属于 RISC”	“只属于 CISC”	现代实现可以混合

RISC/CISC 描述 ISA 设计取向，硬布线/微程序描述控制实现。两组概念相关，但不存在一一绑定。

9 提交：写使能才是架构状态变化的闸门

一个值在 ALU 输出端稳定，只说明它**已经产生**；被后级路径选中，只说明它可以**到达**；只有在合法时钟边界打开正确状态元件的写使能，它才**提交**为新状态。

Produced ≠ Available at Destination ≠ Architecturally Committed

这一区分提供一个强校验：对照指令 Write Set，检查所有可能写口。

- 应写的位置是否真的打开？
- 不应写的位置是否全部关闭？
- 若发生异常，故障指令是否留下了禁止的部分副作用？
- PC 的正常值、分支值和异常入口是否由互斥条件选择？

精确异常的跨科接口只保留最小要求：异常前较老的架构效果已经成立；故障指令及更年轻指令的禁止副作用没有提交。具体 OS handler 不属于本册。

10 概念边界：相似名词为什么会误导通路题

概念 A	概念 B	真正区别与题目信号	混淆后果
架构状态	≠ 微架构状态	软件契约可依赖前者；IR/MDR 等通常只服务实现。问“不同实现能否不同”时先分层。	把教学寄存器误写成 ISA 必需部件。
ISA 语义	≠ 数据通路	前者规定结果，后者提供实现路径。给指令定义时先写状态差。	用某张通路图反向篡改指令语义。
数据通路	≠ 控制器	前者定义可能的数据移动，后者选择此刻允许的移动。	只连线不检查写使能和冲突。
组合结果产生	≠ 状态提交	前者是线路值稳定，后者需要合法写入状态元件。	认为 ALU 算完就等于指令完成。

概念 A	概念 B	真正区别与题目信号	混淆后果
指令周期	≠ 时钟周期	一条指令可跨多个时钟周期；题目给“周期”时要确认量纲。	CPI、节拍和总时间混算。
微操作	≠ 微指令	微操作是数据动作；微指令是编码若干控制信号的控制存储字。	把动作序列与编码载体混为一谈。
内部 CPU 总线	≠ 系统总线	前者连接处理器内部部件，后者组织芯片/设备事务；模型和仲裁范围不同。	把单总线微操作规则套到外部总线。
硬布线/微程序	≠ RISC/CISC	前一组是实现机制，后一组是 ISA 设计取向。	形成“一一对应”的错误选择题口诀。

11 贯穿母例：在单总线教学机上实现 LOAD

设教学机满足以下明确假设：

- 字节编址，顺序 PC 步长为 L ；
- CPU 内部只有一条总线，同拍唯一源驱动；
- ALU 的一个输入经 Y 暂存，输出经 Z 暂存；
- MAR/MDR 与存储接口交互；
- 存储读完成由题设时序保证或显式等待信号决定。

目标指令为

```
LOAD rd, imm(rs1).
```

11.1 先写语义，不先写节拍

$$EA = R[rs1] + sext(imm),$$
$$D = M[EA],$$
$$R'[rd] = extend(D),$$
$$PC' = PC + L.$$

11.2 再按资源约束调度

一种合法但非唯一的调度是：

阶段	微操作	生产/消费	校验
T_1	$PC \rightarrow MAR, PC \rightarrow Y$	产生取指地址和顺序 PC 输入	同一总线源 PC，可多目标锁存
T_2	$M[MAR] \rightarrow MDR, Y + L \rightarrow Z$	存储返回指令；ALU 产生下一 PC	仅在存储与 ALU 可并行假设下同拍
T_3	$Z \rightarrow PC$	提交顺序 PC	PC 写使能打开
T_4	$MDR \rightarrow IR$	当前指令进入译码状态	不改变通用寄存器
T_5	$R[rs1] \rightarrow Y$	锁存 base	寄存器编号来自指令字段
T_6	$Y + sext(imm) \rightarrow Z$	产生 EA	扩展规则来自 ISA
T_7	$Z \rightarrow MAR$	发出数据地址	仍未得到 load 数据
T_8	$M[MAR] \rightarrow MDR$	数据返回	若存储未 ready，必须等待
T_9	$MDR \rightarrow R[rd]$	提交目标寄存器	MemWrite 必须关闭

母例故意暴露的错误路径

若在 T_6 就打开目标寄存器写使能，写入的是有效地址而非加载数据；若在 T_8 打开内存写信号，则 LOAD 被错误实现成 STORE。两种错误都能通过“Write Set 反查”提前发现。

固定九拍不是知识本体。换成双总线、多端口寄存器堆、独立 PC 加法器或不同存储握手后，调度会改变；状态差、依赖和提交边界不变。

12 怎样为新指令扩展数据通路

题目要求“加入某条指令”时，按以下顺序工作：

- 1. 写该指令的 Read Set、Derived Values、Write Set 和 Next PC；
- 2. 将每个中间值标成“生产者 → 消费者”；
- 3. 在现有通路上寻找路径，缺路才新增连线、MUX 或功能部件；
- 4. 检查端口和资源冲突，决定是否新增阶段或复制资源；
- 5. 补充控制信号真值和控制器状态转移；
- 6. 重新计算关键路径或新 CPI；
- 7. 用 Write Set 和异常路径验证没有多余副作用。

最小硬件原则

“支持新指令”不等于先加一个新部件。若现有 ALU、MUX 和暂存寄存器能通过新的调度完成语义，新增控制状态可能比复制硬件更合理；若性能约束不允许多拍，再讨论复制资源。先证明缺什么，再加什么。

13 性能：从路径和资源生成数字

13.1 关键路径

关键路径不是“部件延迟全部相加”。只有同一周期内、沿真实依赖串联的组合部件才累加；互相并行的路径取最大值。

$$T_{comb,max} = \max_{p \in \text{legal paths}} \sum_{u \in p} T_u.$$

13.2 资源复制与复用

设计动作	获得的能力	支付的代价
增加独立 PC 加法器	PC 更新可与主 ALU 工作并行	面积、功耗、布线
增加寄存器堆端口	同拍读取/写入更多值	阵列面积与访问延迟
加入更多 MUX 输入	复用现有功能部件	选择控制与关键路径变长
加入临时寄存器	跨拍保存中间值、缩短周期	状态和控制阶段增加
复制存储端口	消除取指/数据访问冲突	存储结构复杂度和成本

13.3 题目成本模型

遇到性能题先写：
what overlaps? what serializes? what is the unit?

再计算时钟周期、每类指令 CPI、指令混合和总执行时间。若题目没有给 setup、skew 或控制延迟，只能按题设简化，不能暗中加入另一套口径。

14 做题控制协议

CO-03 七步协议

1. **Delta**：写指令前后允许变化的架构状态；

2. **Values**：列 Read Set、Derived Values、Write Set；

3. **Graph**：标出每个值的生产者和消费者；

4. **Path**：沿现有部件连可行路径，选择点才放 MUX；

5. **Schedule**：按依赖、资源、时序和提交四道门拆拍；

6. **Control**：为每拍写唯一驱动、锁存目标、选择信号和条件；

7. **Verify**：用 ISA Write Set、关键路径和异常边界做独立反查。

14.1 题目信号与第一动作

题目信号	第一动作
“补全数据通路/支持新指令”	写 before/after 状态差，不先画线
“哪些微操作可并行”	列源、目标、依赖和共享资源
“求控制信号”	先写当前拍的数据动作，再映射 Enable/Select/Command
“单周期与多周期比较”	写 $IC \times CPI \times T_{clk}$ ，统一成本口径
“关键路径”	只沿同一周期真实串联路径求和，再取最大
“发生异常/中断”	先查哪些架构写口尚未或不得提交

15 高频失效模式

1. **从图开始**：记得部件，却不知道为什么需要它。修复：先写 State Delta。
2. **把箭头当并行**：忽略总线、端口或 ALU 冲突。修复：逐拍列资源占用。
3. **把产生当提交**：ALU 输出有值就认为寄存器已更新。修复：追踪写使能和时钟边界。
4. **把实现当 ISA**：认为 MAR/MDR、九节拍或固定 PC 步长普遍存在。修复：给所有实现细节加前提。
5. **只比 CPI**：宣称单周期 CPI 低所以更快。修复：比较完整 CPU time。
6. **RISC/CISC 口诀化**：把 RISC 与硬布线、CISC 与微程序绝对绑定。修复：分开 ISA 与实现两条轴。
7. **越过本册边界**：在本册展开 Cache miss、Page Fault 或操作系统 handler。处理方法：只保留存储接口和精确提交要求。

16 本册与指令语义的接口

从状态差到控制信号

本册把一条指令的架构状态差转换为值依赖、数据通路、微操作、资源调度和控制字；结果是一条可计时、可检查的单指令路径。多条指令同时执行时，还要额外检查 ready/need 和 flush 条件。

本册的 Cost 不是“控制信号越多越慢”，而是由关键组合路径、时钟周期、资源复制/复用、存储端口冲突和单/多周期调度共同决定。比较实现时必须统一 IC、CPI 与 clock period。

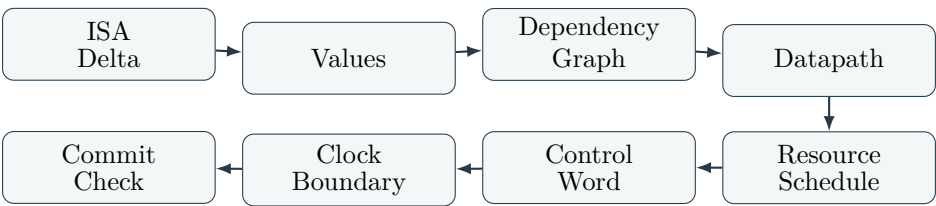
17 考纲映射与接口

传统考点	本册位置	下游接口
CPU 功能与基本结构	架构/微架构状态、最小构件	CO-B01
指令周期与微操作	值依赖图、四道可行性 Gate	CO-I01
单总线数据通路	唯一驱动与多目标锁存	具体教学机题
单周期/多周期 CPU	同一依赖图的不同调度	CO-04 流水线
硬布线/微程序控制	条件到控制字和下一阶段	CO-02 ISA Use
关键路径与性能	真实串联路径和 CPU time	Subject Rules
异常时的状态边界	Produced/Available/Commit	X-B01、CO-04

18 一页压缩：忘掉整张图后怎样重建

一句话

CPU 数据通路与控制的任务，是把 ISA 的状态差翻译成值依赖图，再在有限资源和时序约束下调度微操作，用控制字只在合法边界提交允许的状态变化。



六个重建问题

- 1. 指令允许哪些架构状态改变，哪些绝不能改变？
- 2. 每个所需值在哪里产生，又在哪里被消费？
- 3. 现有通路缺的是路线、选择器、端口还是功能部件？
- 4. 哪些动作因依赖、资源或关键路径必须拆拍？
- 5. 当前阶段应打开哪些 Enable、Select 和 Command？
- 6. 值何时产生、何时可达、何时才真正提交？

边界说明

本册的九节拍 LOAD 轨迹只是一个带明确假设的教学模型；真实处理器可以合并或拆分阶段。多核、Cache、虚拟存储、完整 I/O、中断服务和 ABI 属于其他层次，本册只说明它们在何处给控制器输入或形成异常，不把某种实现写成普遍定律。

19 补充细节：从 CPU 功能到控制器状态机

CPU 结构、指令周期、单周期/多周期、单总线、硬布线和微程序都可以压回同一条生成链：

Architectural State Delta → Value / Resource Dependency → Micro-operation Schedule → Control Word / Next S

“CPU 有哪些部件”只是结果，不是起点；每一个寄存器、MUX、端口和控制信号都必须能回答“它承载哪一个值、阻止哪一种冲突、在哪个时钟边界锁存”。

19.1 CPU 的功能由程序状态变化反推

从存储程序模型出发，CPU 必须完成：取指、解释、执行、访问存储、更新 PC 和处理异常/中断。由此推导最小部件：

- PC 保存下一条指令地址，并能顺序加法、分支重载或异常入口重载；
- IR 保存当前指令字段，使译码和后续微操作在取指完成后仍有稳定输入；
- 寄存器堆提供多个读端口和受时钟控制的写端口；
- ALU/移位器产生地址、算术逻辑结果、比较结果和下一 PC 候选；
- MAR/MDR 或等价的存储接口寄存器隔离 CPU 内部时序与存储器响应；
- 控制器根据 opcode、条件码、ready/ack 和当前阶段生成控制字与下一阶段。

寄存器名称不是跨架构定律。PC、IR、MAR、MDR、ACC、PSW、通用寄存器和流水寄存器是常见教学抽象；某个真实实现可以合并、复制或隐藏它们，只要架构状态轨迹不变。

对象	保存的状态	典型生产者/消费者	不能直接推出
PC	下一取指地址	加法器、分支目标、异常入口	必须是固定宽度或显式寄存器名
IR	当前指令编码	存储接口产生，译码器消费	所有机器只有一个 IR
寄存器堆	架构源/目的值	指令读端口、写回端口	读写一定同拍完成
MAR/地址寄存器	当前存储请求地址	EA/PC 形成器产生	物理地址而非虚拟地址
MDR/数据寄存器	存储读回或写出数据	存储器/总线与写回逻辑	一定需要独立硬件
PSW/flags	条件和控制状态	ALU/异常逻辑	对所有 ISA 都同一组标志

19.2 四个子周期与控制器的阶段证据

- 传统指令周期常拆成取指、间址、执行和中断四个子周期。它们是行为阶段，不是固定节拍数：
1. 取指：按 PC 访问存储器，IR 接收指令，PC 形成顺序候选；
 2. 间址：若操作数采用间接方式，先取得有效地址或完成地址链；

3. 执行：完成算术逻辑、访存、控制转移或设备请求；
4. 中断：在允许的边界保存必要架构状态、识别来源、装入入口并转入 handler。
- 同一子周期可拆成多个微操作，也可以在多端口或更宽数据通路中合并。控制器识别当前阶段通常依赖状态寄存器、IR opcode、条件码和外部 ready/interrupt 信号；不能从“四个名字”反推只有四个时钟周期。

19.3 单总线数据通路：唯一驱动与中间寄存器

单总线结构在一个时刻通常只允许一个源驱动总线，但可让一个或多个目标在时钟边界锁存该值。因而

$$\text{Bus Source} \rightarrow \text{Bus} \rightarrow \{\text{Destination Enables}\}$$

必须满足“一个驱动、多个合法接收”的规则。若同拍需要先保存总线值、再让 ALU 运算并把结果送回总线，通常引入 Y（ALU 输入保持）和 Z（ALU 输出保持）等中间寄存器：

$$R_i \rightarrow Y, \quad R_j \rightarrow \text{Bus}, \quad \text{ALU}(Y, \text{Bus}) \rightarrow Z, \quad Z \rightarrow R_k.$$

Y/Z 不是 ISA 状态，它们解决单总线下“同一条总线不能同时提供两个输入和一个输出”的资源冲突。若题目给出不同的总线数量或 ALU 多输入结构，应重新推导微操作，不机械复制 Y/Z。

单总线微操作的四问

每一拍依次检查：谁驱动总线？谁锁存总线？ALU 两个输入在拍开始时是否已经稳定？写使能在时钟边界是否只提交题目允许的架构状态？若任一答案不清楚，不能把两个箭头写在同一拍。

19.4 单周期与多周期：同一依赖图的两端调度

单周期把一条指令的取指、译码、地址形成、访存、执行和写回放进一个长周期， $CPI = 1$ 但时钟周期必须覆盖最慢路径；共享资源若在同拍被多次需要，必须使用双端口、独立存储、复制部件或题设允许的组合逻辑。单周期并不必然等于哈佛结构。

多周期把同一依赖图拆到多个较短周期中，让一个部件在不同时刻复用。典型阶段是 IF、ID、EX、MEM、WB，但不同指令可以跳过不需要的阶段；因此每条指令的周期数不同，控制器必须保存阶段状态，并用中间寄存器保持已产生但尚未提交的值：IR、MDR、A、B、ALUOut 等都是这种需求的实现例。

比较轴	单周期	多周期	真正要统一的成本口径
时钟周期	覆盖全指令关键路径	覆盖单个阶段关键路径	T_{clk} 由真实最长组合路径决定
CPI	通常 1	随指令类型变化	不能只比较 CPI
资源	同拍需求多，常需复制/多端口	跨拍复用，硬件较少	资源成本与时序成本交换
控制器	当前 opcode 直接组合控制	状态机/微程序序列	控制复杂度也要计入实现
整体性能	$IC \times 1 \times T_{long}$	$IC \times CPI_{avg} \times T_{short}$	统一程序和完整 CPU time

19.5 硬布线控制：从条件表生成控制逻辑

硬布线控制器把 opcode、阶段状态、条件码和外部信号送入组合逻辑，直接生成各个数据通路控制线以及下一状态。设计过程可以写成四张表：当前状态/输入、允许的微操作、输出控制字、下一状态。它的优势是常见路径延迟短；代价是指令集或阶段改变时逻辑网络复杂、验证和扩展困难。

“控制信号多”不等于“硬布线一定慢”。真正的成本来自关键控制路径、逻辑深度、扇出和状态机复杂度；题目要求补全控制信号时，先写本拍数据动作，再将动作映射为 PCWrite、IRWrite、RegWrite、MemRead、MemWrite、ALUSrc、ALUOp、通路选择等信号。不同教材的信号名可以不同，真正稳定的是这些信号所允许的数据动作。

19.6 微程序控制：把控制字放入控制存储器

微程序控制把一条机器指令展开为一串微指令。典型结构包含控制存储器、微地址寄存器（uPC）、微指令寄存器、微指令地址形成逻辑和条件测试逻辑。微指令至少分为：

- 操作控制字段：并行或编码选择数据通路控制信号；
- 顺序控制字段：给出下一微地址、条件分支方式或映射入口；
- 判定/测试字段：选择 opcode、条件码、存储器 ready 或中断请求作为下一地址依据。

操作控制字段可以采用水平型直接编码、垂直型字段编码或混合编码。直接编码并行度高但微指令长，字段编码节省控制存储器但同一字段内不能同时打开互斥动作。微地址可以顺序加一、条件分支、按 opcode 映射或跳回取指入口；“微程序控制就是顺序执行微指令”是不完整模型。

控制实现	控制字来源	优点	边界
硬布线	组合逻辑/状态译码	快、常规路径直接	扩展与验证成本高
微程序	控制存储器中的微指令	复杂 ISA 易组织、易修改	取微指令和控制存储器有额外开销
混合实现	热点硬布线，复杂路径微码	平衡延迟和灵活性	必须说明哪些路径采用哪种实现

19.7 异常、中断与精确提交：控制器必须保护状态边界

同步异常由当前指令或当前执行阶段触发，外部中断由设备/外部事件触发；二者都可能让控制器离开正常下一状态，但返回语义不同。故障类异常通常允许处理后重新执行当前指令，陷阱/终止类语义由 ISA 另定；外中断一般在允许的指令边界响应。硬件检测到条件不等于 OS 已经执行 handler，也不等于所有年轻指令都可以保留。

流水或多周期实现中，精确状态要求：已提交的老指令效果保留，异常指令及其后的未提交效果不可见。CO-03 只负责单指令内部的写使能和异常入口边界；多条指令的顺序、flush 和恢复由 CO-04 继续拥有。

19.8 教学机与模拟器应怎样使用

遇到任何单周期、多周期、单总线或微程序图，先把图中的每个开关翻译成：当前状态、驱动源、锁存目标、控制字段和下一阶段；不要把某张可视化图直接当作唯一实现。推荐的三次观察顺序是：

1. 只看一条 ADD/LOAD，记录每个值的生产者、消费者和写回边界；
2. 改变总线/端口或执行模式，观察哪些动作必须拆拍、哪些动作可以并行；
3. 注入 memory not-ready、异常或分支条件，观察控制器如何保持/清空状态并选择下一入口。

模拟器验证的对象是“模型能否预测可见状态”，不是“界面是否长得像教材图”。

20 微架构陌生题反向重建协议

遇到没有见过的数据通路或控制器图，按以下顺序：

1. 写 ISA 状态差，列 Read Set、Derived Values、Write Set 和 Next PC；
2. 把每个值放到生产者—消费者图中，标出需要的存储/ALU/总线资源；
3. 逐拍检查数据依赖、共享资源、关键路径和写使能四道 Gate；
4. 若单总线一次只能有一个驱动，加入 Y/Z 或拆拍；若多端口已给出，重新判断可并行动作；
5. 将当前拍动作映射为控制字，并说明下一状态如何由 opcode/条件/ready 决定；
6. 最后检查异常、写回和提交边界，再把多指令重叠问题交给 CO-04。

CO-03 的扩充停止条件

读者能从陌生 ISA 状态差重新生成最小数据通路、指出为何需要某个中间寄存器或控制信号、排除一个资源冲突，并写出单/多周期下的合法提交边界时，CO-03 的机制层完成。具体流水线 hazard、Cache/TLB miss 和 OS handler 不在本册重复拥有。